Introduction to STM32 Peripherals

Ankit Srivastava

EE2028

Unless stated otherwise, for the second part of the module,

‘STM32 Chip’ = STM32L4S5VIT6

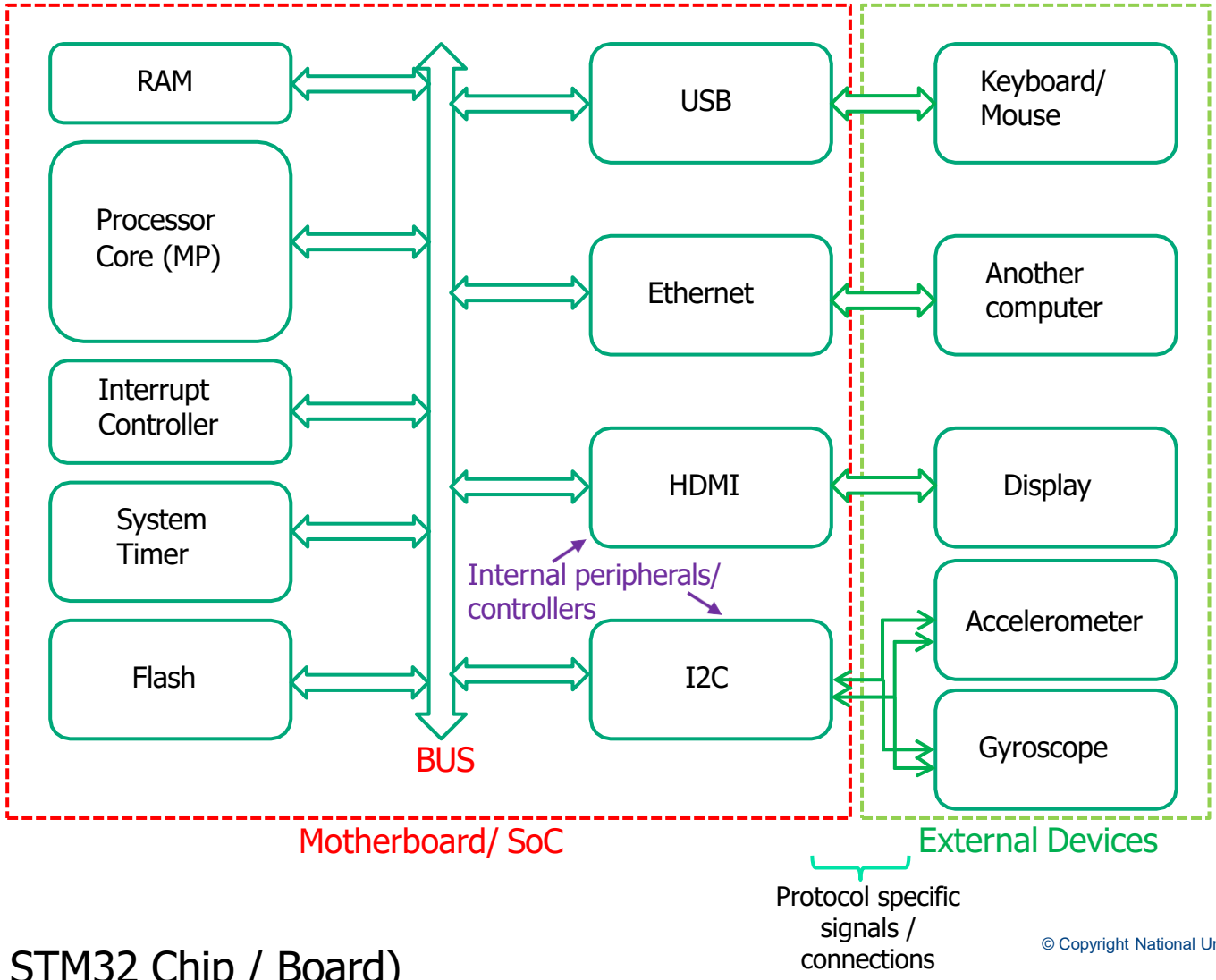
‘Board’ = B-L4S5I-IOT01A

Acknowledgement: Slides adapted from Dr Rajesh Panicker

OVERVIEW

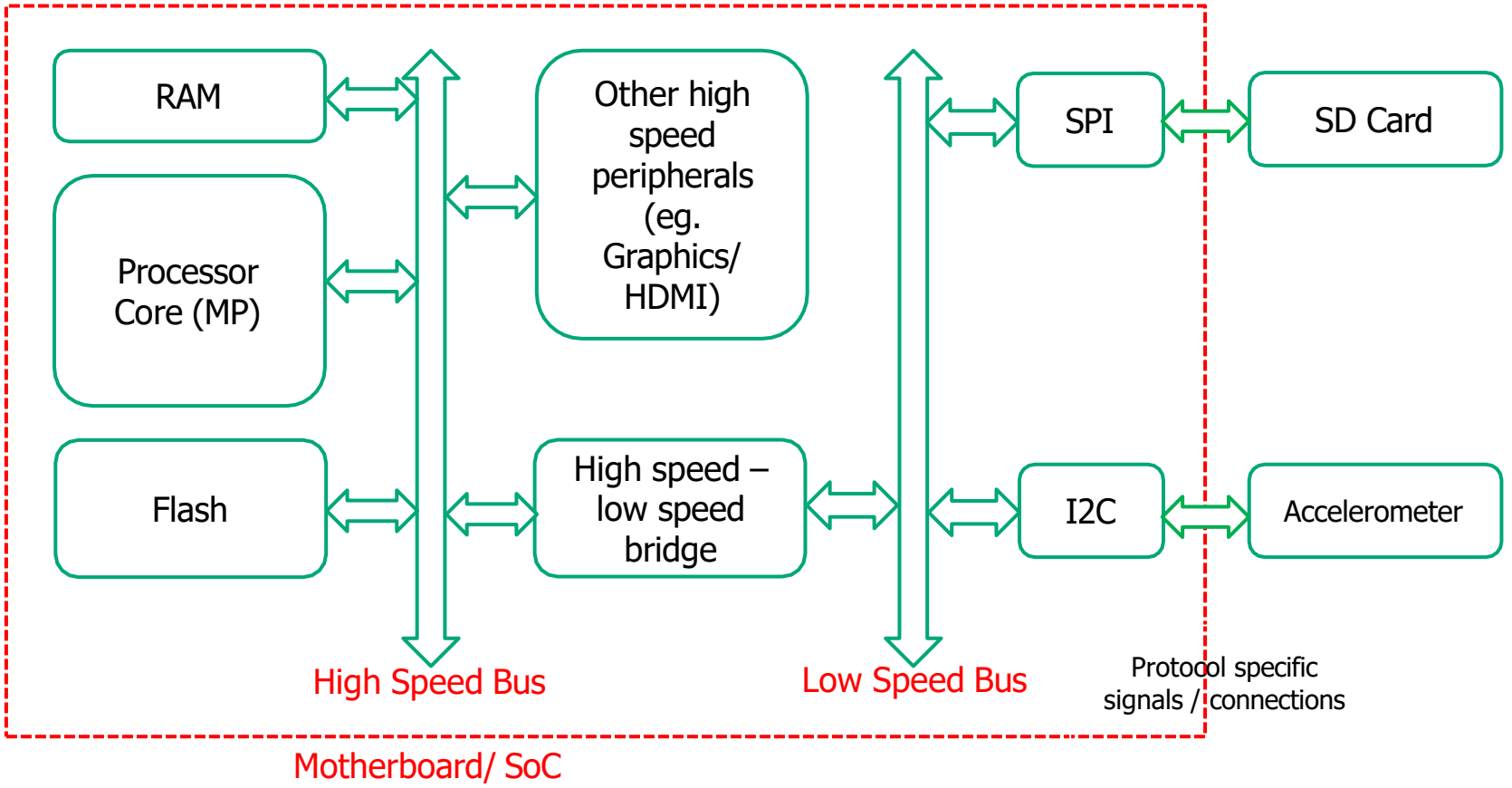
- What is a Computer System?
- STM32 Microcontroller and Development Board
- Accessing STM32 Peripherals via Libraries

What is a Computer System?

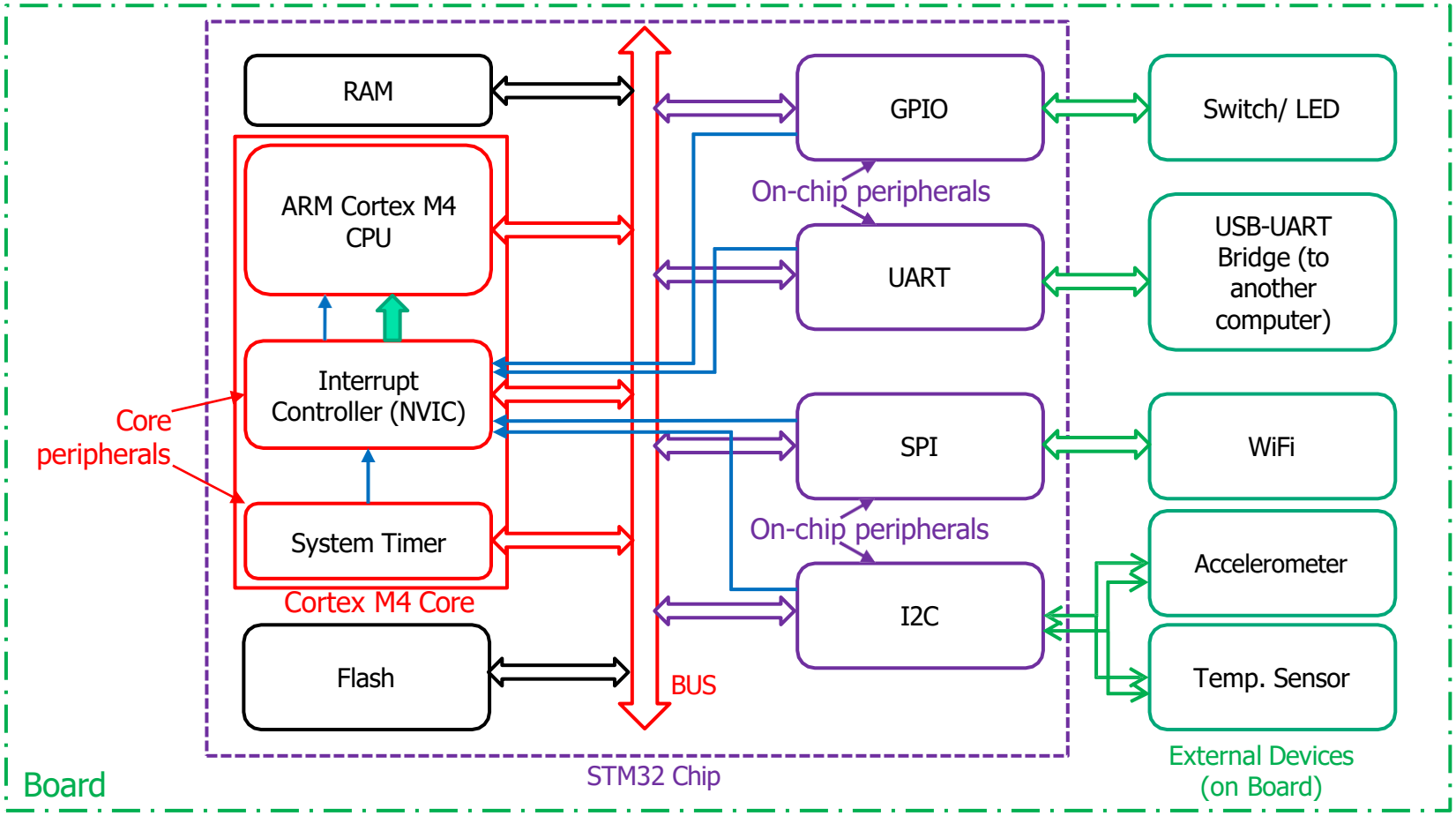


(Not specifically STM32 Chip / Board)

MULTI-TIERED BUS ARCHITECTURE

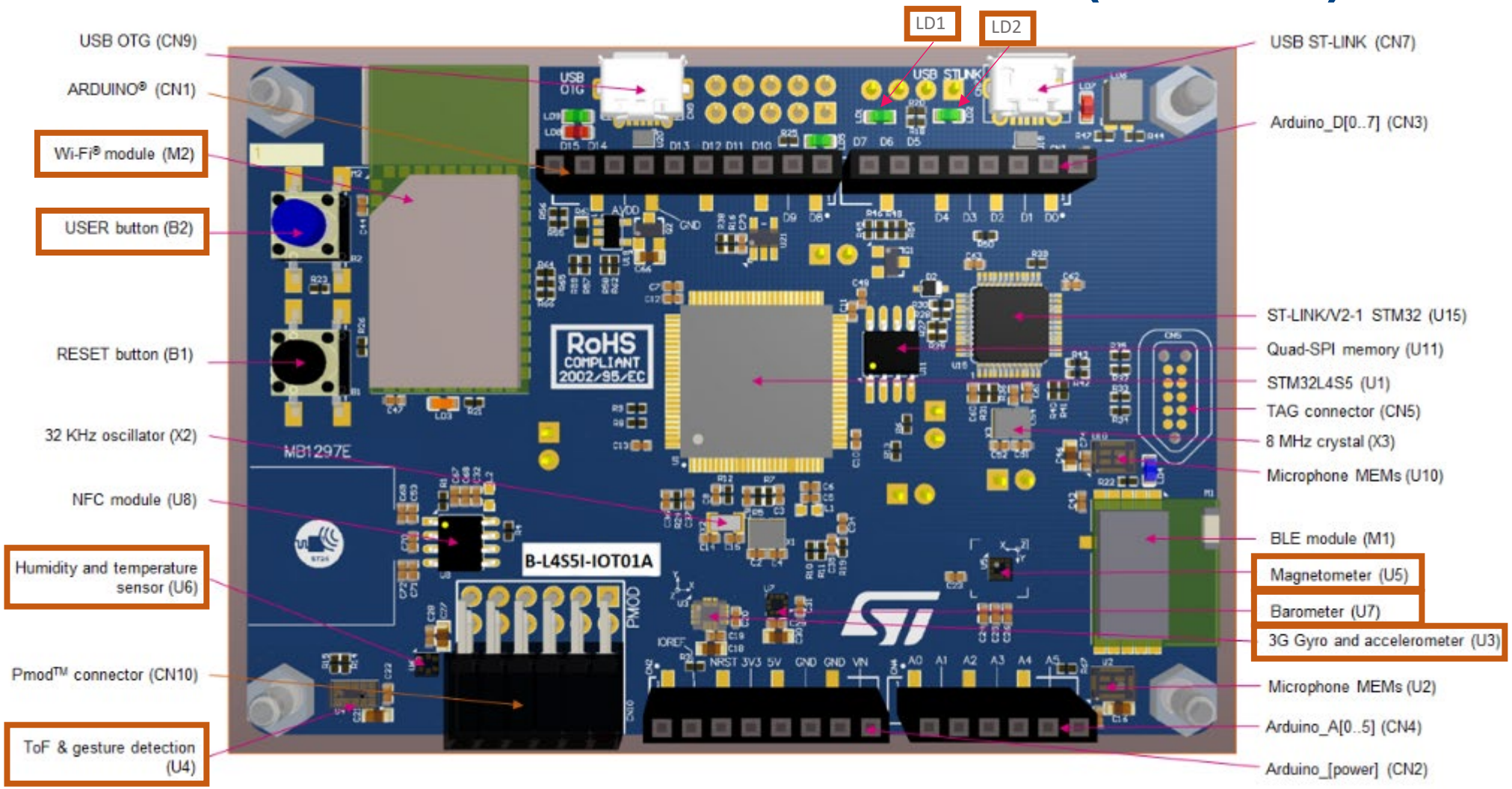


THREE-TIER COMPOSITION OF STM32 BOARD



- **Core peripherals** from ARM, tightly integrated with Cortex M4
- **On-chip peripherals** from STM. Part of STM32 Chip
- **External devices / peripherals** mostly from STM on the Board

STM32 Chip and Development Board (Partial)



Refer to UM2708: STM32L4+ series Discovery kit, Fig 3 (pg. 8)

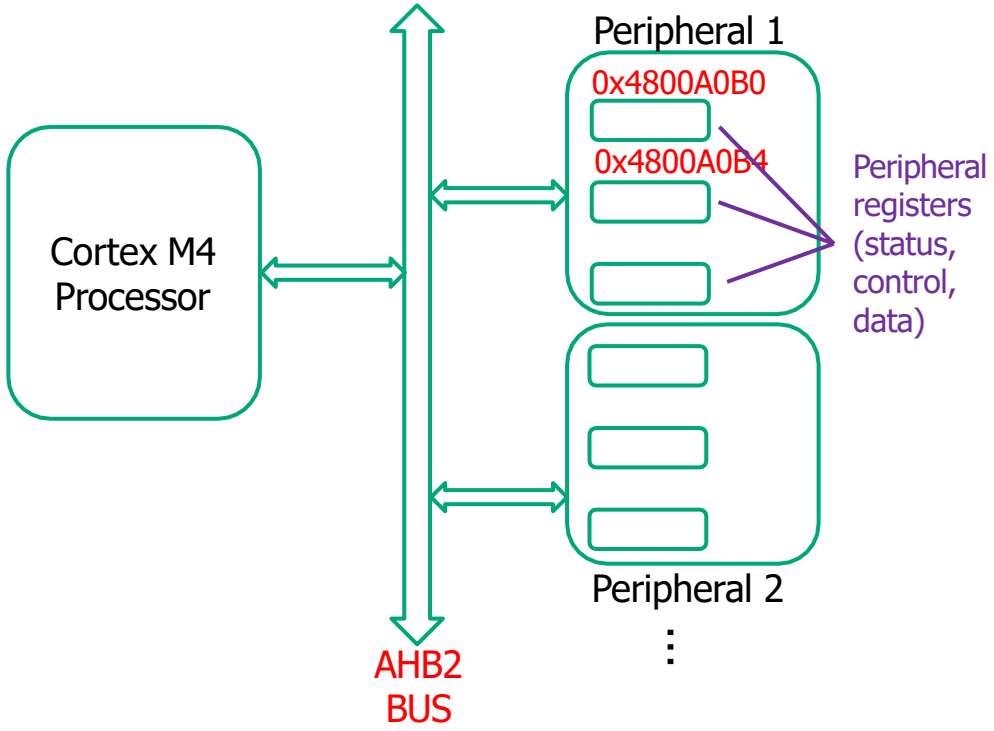
Programming Hardware Peripherals via Registers

Software interacts with the hardware by reading/writing peripheral registers

- Details of the registers and what happens when you write them / what you get when you read them can be found in the appropriate datasheet

Broadly 3 categories:

- Status registers - *read* by the processor to know the status of the hardware
- Control registers - *written* by the processor to give commands / control signals
- Data registers - *read* (for input) and/or *written* (for output) to perform actual transfer of data
- Some functionalities could be combined in some registers, e.g., control cum status registers



Peripheral registers are accessed like memory – using LDR/STR equiv.

NAVIGATING STM32 DOCUMENTATION

Reference Manual for ARMv7-M

↓

Datasheet for STM32L4+ Processor Series

↓

Datasheet for specific SoC: STM32L4S5VIT6

↓

User Manual for Board: B-L4S5I-IOT01A

↓

Datasheets of specific Devices on Board (self-exploration)

Name	Description	Link
ARMv7-M Architecure Ref Manual.pdf	Details about the processor. Assembly language instructions etc.	
STM32L4x5-6 Ref Manual_RM0351.pdf	Generic details about STM32L4x5-6 System-on-Chip. Description, configuration, and operation of the various peripherals such as GPIO, I2C etc.	
STM32l475vg Datasheet.pdf	Finer details specific to STM32L475VG System-on-Chip. Alternate functions for each pin etc.	
B-L475E-IOT01A_user_manual_UM2153.pdf	Details about the B-L475E-IOT01A board. Board layout and location of various on-board devices such as sensors, description of the devices, pin connections, schematics.	

Refer to Canvas > EE2028 > FIles > Labs > Data sheet

Peripheral Abstraction via Software Libraries

Libraries:

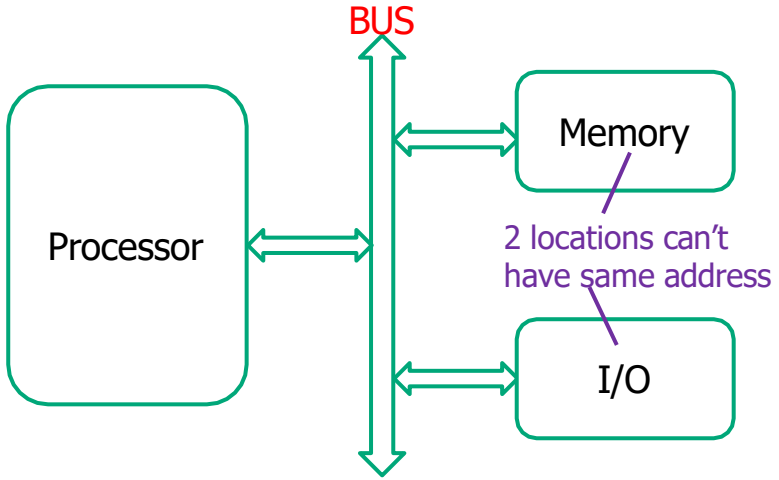
- are collection of **driver functions or APIs** to access a peripheral
- **encapsulate/abstract** the process of reading/writing peripheral registers
- internally **use pointers to peripheral registers** to accomplish reading/writing
  - pointers hold addresses of peripheral register
  - pointers are implemented as **LDR/STR instructions** or equivalent
- allow **ease of programming** (higher layer of abstraction - no need to reinvent the wheel)
- provide **portability** - we just need to recompile/re-link with the new library, user program will need minimal/no changes

CMSIS: functions for core peripherals (NVIC/Interrupts, SysTick etc.)

HAL: functions for on-chip peripherals (GPIO, I2C, UART, SPI etc.)

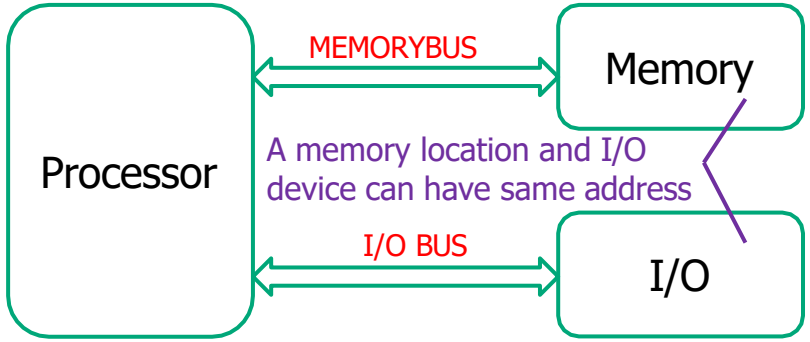
BSP: functions for external devices/peripherals on the baseboard (temperature sensor etc.)

Peripheral Access (MMIO vs PMIO)



Memory-mapped IO (MMIO) : the addresses for peripheral registers shares the address space with the memory

- LDR/STR (or pointers) used for IO



Port-mapped IO (PMIO) : separate address spaces for memory and peripherals

- Separate instructions (eg. IN/OUT in Intel 32) required for IO

Conclusion

- What is a Computer System?
  - Processor, Memory, Peripherals
  - Muti-tiered bus architecture
- STM32 Microcontroller and Development Board
  - Three-tiered composition
    - Cortex-M4 Core, STM32 Chip, Development Board
- Accessing STM32 Peripherals via Libraries
  - CMSIS, HAL, BSP
  - MMIO vs PMIO